

evaluation02_v1b_CORR

November 1, 2022

1 Evaluation n°2

Dans cette évaluation, vous aurez à traiter les chapitres suivants: * While * For

Pour répondre à cette évaluation: * Sauvegarder régulièrement votre travail. * Compléter les programmes ou répondez aux questions dans les cellules appropriées * Vous NE MODIFIEREZ PAS ce qui est déjà écrit dans les cellules. * Certaines cellules serviront à tester vos fonctions. Vous pouvez vous en servir * A la fin du devoir, vous sauvegarderez votre fichier. * Vous rendrez votre fichier via l'ENT / travail à faire

Pour chacun des questions suivantes, compléter les fonctions de manière à ce qu'elles respectent le docstring donné

```
[1]: ## Question 1
def fonction1(n):
    '''
    Fonction renvoyant la somme des n premiers entiers naturels.
    Vous utiliserez obligatoirement une structure FOR
    Exemple si n = 6: somme = 1 + 2 + 3 + 4 + 5 + 6
    : param (int): un entier strictement supérieur à 0
    : return (int): un entier
    >>> fonction1(6)
    21
    >>> fonction1(9)
    45
    '''
    assert type(n) == int , ' n doit etre un entier'
    assert n > 0 , 'n doit etre strictement positif'

    ### A compléter
    somme = 0
    for i in range(1,n+1):
        somme = somme + i

    return somme
```

Attention: * La somme doit être initialisée à 0 * Il faut balayer (itérer) sur les valeurs 1 à n. C'est à dire un range(1,n+1) * La somme doit être incrémentée de la valeur de i à chaque tour de boucle.

[]:

```
[2]: ## Question 2.
### Vous complèterez également les asserts de manière à respecter le docstring
def fonction2(a,b):
    '''
        Fonction effectuant des soustractions successives et renvoyant la première
        ↪ valeur strictement inférieure à 0
        Vous utiliserez obligatoirement une structure WHILE
        Exemple: a = 2 et b = 9
        9 - 2 = 7 supérieur à 0
        7 - 2 = 5 supérieur à 0
        5 - 2 = 3 supérieur à 0
        3 - 2 = 1 supérieur à 0
        1 - 1 = -1 inférieur à 0
        La valeur renvoyée est -1
        : param (int) avec a et b strictement positifs et a < b
        : return (int)
    >>> fonction2(2,9)
    -1
    >>> fonction2(2,10)
    -2
    '''

    ### A compléter avec les asserts
    assert type(a) == type(b) == int
    assert 0 < a < b

    ### A compléter avec le programme
    valeur = b
    while valeur >= 0:
        valeur = valeur - a

    return valeur
```

Attention: * Il faut vérifier que les types de a et de b sont des entiers AVANT de comparer les valeurs * La condition d'arrêt est `valeur < 0` donc la condition WHILE est bien `valeur >= 0` * La valeur b est copiée de manière à ne pas être modifiée * A chaque tour de boucle on soustrait la valeur a * Dès que la condition est fausse, on sort de la boucle et on retourne la valeur

```
[3]: ## Question 3.
def fonction3(a):
    '''
        Fonction qui renvoie le premier chiffre impair dans une chaîne de chiffres
        Si la chaîne de chiffres ne comporte que des chiffres pairs, la valeur
        ↪ renvoyée est un booléen False
        Vous êtes autorisé à utiliser plusieurs "return"
        : param (str) un mot de n chiffres. Chaque caractère est un chiffre.
    '''
```

```

: return (str)
Exemple: mot = '422681654' renverra '1' car c'est le premier caractère
↳ impair
>>> fonction3('1344')
'1'
>>> fonction3('4428261')
'1'
>>> fonction3('2222222222')
False
'''

### A compléter avec les asserts
assert type(a) == str
for chiffre in a:
    assert chiffre in '0123456789'

### A compléter avec le programme
for chiffre in a:
    if int(chiffre) % 2 == 1:
        return chiffre
return False

```

Attention: * la variable a est un str. On peut donc itérer sur a avec la commande FOR * chaque chiffre est un str il faut donc le convertir en int afin de tester s'il est pair ou impair * Si le chiffre est impair il est retourné * Si le chiffre est pair on passe au suivant * Si aucun chiffre n'est impair on renvoie False

```

[4]: ## Question 4.
def fonction4(nombre):
    '''
    Fonction renvoyant le nombre de chiffres pairs contenus dans un nombre
    ↳ passé en paramètre
    : param (int) une valeur strictement positive
    : return (int)
    Exemple: nombre = 467734. Il y a 3 chiffres pairs: 4, 6 et 4
    La valeur renvoyée est donc 3
    >>> fonction4(3346758890)
    5
    >>> fonction4(1234567890)
    5
    '''

    ### A compléter avec les asserts
    assert type(nombre) == int
    assert nombre > 0

    ### A compléter avec le programme
    ## Version 1

```

```

count = 0
while nombre > 0:
    if (nombre % 10) % 2 == 0:
        count = count + 1
    nombre = nombre // 10
return count

## Version 2
count = 0
nombre = str(nombre)
for chiffre in nombre:
    if int(chiffre) % 2 == 0:
        count = count + 1
return count

```

[]:

```

[5]: ## Question 5
def fonction5(a,b):
    '''
    Fonction renvoyant la somme des entiers naturels compris entre a et b
    Vous reutiliserez la fonction1 définie plus haut.
    Exemple: Si a = 3 et b = 6 alors la fonction calculera la somme:
    somme = 3 + 4 + 5 + 6
    : param (int) a et b deux entiers tels que a < b
    : return (int)
    >>> fonction5(3,6)
    18
    '''

    ### A compléter avec le programme
    somme = fonction1(b) - fonction1(a-1)
    return somme

```

[]:

1.1 Question 6

Pour valider un code d'entrée de 5 chiffres (ne commençant pas par 0), on utilise la technique suivante:

- On calcule la somme des 4 1er chiffres
- Si le résultat est égal au 5ème chiffre, le code est valide.
- Sinon le code est invalide

Exemple :

99999 ---> 9+9+9+9 = 36 ---> 6+3 =9 qui est égal au 5ème chiffre (code valide)
 10013 ---> 1+0+0+1 = 2 qui n'est pas égal à 3 (code invalide)

1. Ecrire une fonction `somme_digits(valeur)` qui renvoie la somme des chiffres contenus dans la chaîne de caractère `valeur`
2. Ecrire une fonction `code_est_valide(nombre)` qui renvoie `True` si le code est valide est `False` sinon. Vous réutiliserez la fonction `somme_digits`

```
[6]: def somme_digits(valeur):
    ''' Effectue la somme des chiffres contenus dans la chaîne de caractère_
    ↪valeur
    : param (str) une chaîne de caractères contenant des chiffres
    : return (str)
    >>> somme_digits('123')
    '6'
    >>> somme_digits('9999')
    '36'
    '''

    ## A compléter avec le programme
    somme = 0
    for chiffre in valeur:
        somme = somme + int(chiffre)
    return str(somme)
```

```
[ ]:
```

```
[7]: def code_est_valide(nombre):
    ''' Effectue la vérification de la validité du code
    : param(str): un code de 5 chiffres
    : return (bool)
    >>> code_est_valide('99999')
    True
    >>> code_est_valide('10013')
    False
    '''

    ## A compléter avec le programme
    cle_de_controle = nombre[4]

    valeur_a_analyser = ''
    for i in range(4):
        valeur_a_analyser = valeur_a_analyser + nombre[i]

    # 2eme version: (en utilisant le slicing)
    # valeur_a_analyser = nombre[1:5]

    while len(valeur_a_analyser) != 1:
        valeur_a_analyser = somme_digits(valeur_a_analyser)

    return (valeur_a_analyser == cle_de_controle)
```

```
[34]: code_est_valide('99999')
```

```
[34]: True
```

```
[8]: #####
##### doctest : test la correction des fonctions
##### Ne pas modifier cette cellule
#####

if __name__ == "__main__":
    import doctest
    doctest.testmod(verbose = False) # verbose == False si seules erreurs
    ↪ détaillées
```